SemL0: Query Signatures for Dynamic Property-Graph Storage

Author Name(s)
Institution
Country
email@example.com

ABSTRACT

Typed-neighbor reads already reveal source labels, edge types, direction, and other graph semantics, yet LSM storage usually admits files and schedules compaction using only keys, levels, and bytes. This mismatch creates semantic read amplification and lets compaction erase metadata that once proved a file irrelevant. We present SemL0, a lifecycle-oriented prototype in which query signatures become persistent storage-admission evidence. Its contract permits skipping under exact evidence or a safe conservative over-approximation, but never under unknown evidence. A file budget selects which label/type groups receive degree-exact L0 partitions; unselected groups retain schema-level pruning.

Stage-specific experiments on LDBC SNB data show the trade-off. At SF10, budget-64 uses 13.3 GiB, 444 L0 files, and 2.26 GiB peak import RSS, versus 14.6 GiB, 737 files, and 8.38 GiB for full semantic materialization. At SF100, the corresponding RSS values are 2.21 and 118.03 GiB. Controlled compaction and a real 1.09-billion-edge SF30 run show that semantic merge retains the pruning surface, while naive merge destroys it; the SF30 read consequence is a metadata-replay proxy, not a full body-read speedup. The evidence is decomposed by lifecycle stage rather than claimed as one end-to-end validation.

KEYWORDS

dynamic graphs, LSM-tree, property graph, storage layout, compaction

1 MOTIVATION AND THESIS

Property-graph reads ask for semantic neighborhoods, for example outgoing `KNOWS` edges from a `Person`, rather than an undifferentiated key range. LDBC SNB makes typed neighborhoods central to its workload [2]. Dynamic graph stores nevertheless benefit from LSM organization: updates enter a mutable component, flush into immutable segments, and are rewritten by compaction [1, 3]. The problem is that a selective query may know its edge type while storage metadata cannot prove that most L0 segments are irrelevant. It must then admit and inspect them.

Compaction makes this more than a query-time filtering problem. Suppose three segments represent Person acquaintances, Person-to-post likes, and Forum membership as separate exact partitions. A naive merge can preserve every logical edge while emitting a mixed segment. Future queries lose the proof required to skip unrelated bytes. Thus, compaction is not semantically neutral: it changes the future pruning surface.

SemL0 studies the thesis that query semantics should be a *budgeted storage control plane*. The implementation contributes: (1) a storage signature and an explicit safe-skip contract; (2) a budget algorithm that interpolates between a strong schema layout and full semantic materialization; and (3) semantic-aware compaction that preserves or rebuilds useful evidence. We claim a *lifecycle-oriented design with stage-specific evidence*, not one experiment that validates the complete lifecycle. The current property support is also deliberately narrow: storage admission summarizes property presence, not arbitrary equality, range, or `WHERE` predicates.

2 DESIGN

2.1 Admission evidence

Figure 1 shows the implemented boundary. A graph access signature contains source vertex and label, optional edge type, direction, optional degree class and destination label, optional record timestamp bounds, and an optional property-presence requirement. Its two timestamp fields are segment-admission bounds over records; they are not MVCC snapshots. The read API passes a snapshot separately. Schema names and encodings are likewise resolved by the schema catalog plus each segment's schema epoch; epoch is not a signature field.

Segment metadata carries semantic sets and their completeness. Exact means the recorded set equals the represented set. Conservative means a safe over-approximation: it may add false positives but omits no represented value, so a disjoint over-approximation still proves that a segment can be skipped. Unknown provides no containment guarantee and therefore no semantic pruning. Mixed is different: it is a content/layout state such as a mixed edge-type sentinel or mixed degree classes. It admits every relevant query on that dimension. These rules separate correctness from optimization: coarse evidence causes extra reads, not false negatives.

The property bitmap has two classes: the property must be present, or its absence/default is accepted. Proven absence can prune only a segment without tombstones. Tombstone sensitivity forces a read. Property-value equality has a separate prototype path and does not extend the signature; general value predicates remain future work.

2.2 Budgeted L0 layout

Full semantic layout partitions every group by `(source label, edge type, degree class)`. This can multiply files and metadata. SemL0 instead forms one candidate g per `(source label, edge type)`, measures its bytes b_g, and estimates its degree-exact file cost against the 64 MiB target segment size T:

 17th Annual Conference on Innovative Data Systems Research (CIDR '27). January 24-27, Amsterdam, The Netherlands.

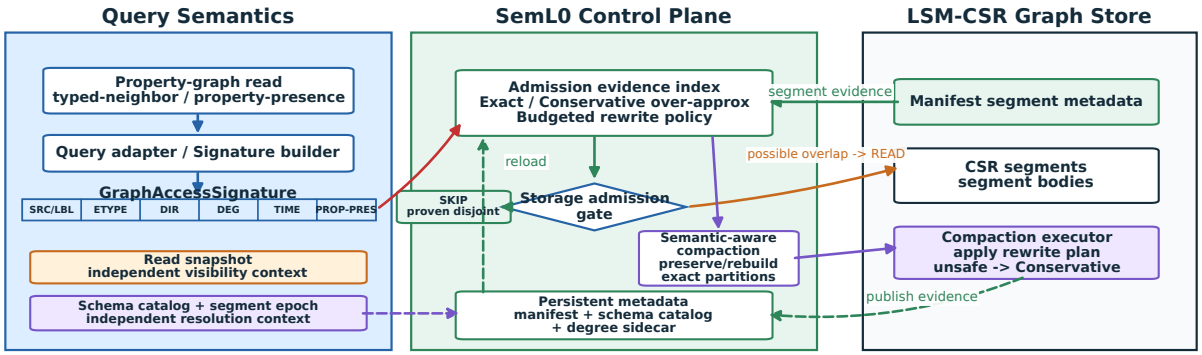

Figure 1: The SemL0 control plane, aligned with the implementation. A `GraphAccessSignature` contains storage-admission facts; the read snapshot and schema/segment-epoch resolution are independent contexts. Exact disjointness, proven absence, or disjoint conservative over-approximations permit SKIP; unknown evidence or possible overlap requires READ. Compaction republishes evidence through the manifest, schema catalog, and degree sidecar.

$$
\begin{aligned}
c_g &= \lceil b_g/T \rceil, \\
f_g &= \sum_{r \in R_g} \min(q_r \max(a_r, 1), 10^6), \\
w_g &= \max(w_g^{cfg}, f_g), \\
s_g &= w_g(b_g/1024)/c_g.
\end{aligned}
\tag{1}
$$

Here R_g contains the observed range snapshots for group g; q_r is a range's query count and a_r is its average candidate-segment pressure. The configured weights are 4.0 for the nine core LDBC edge types, 2.0 for their reverse types, and 0.5 otherwise. Invalid or absent feedback is zero; feedback is capped before taking the maximum. Candidates are sorted by s_g. With file budget B, a candidate is selected only if used + $c_g \le B$; selection increments used by c_g. The W6 budget-64 row sets the byte gate to zero and $B = 64$. Therefore, 64 is the maximum number of *charged additional degree-exact L0 files*, not the total L0 file count and not 64 files per flush.

For a selected candidate, each source is classified by its local neighbor count: Low is 1–16, Medium is 17–1024, and High is above 1024. Degree classes that pass the implementation's per-class benefit gate become separate exact partitions; low-benefit classes remain mixed. An unselected candidate retains its source-label and edge-type partition with mixed degree. It preserves the schema layout's label/edge-type pruning and gives up only the extra degree split. This detail is why budgeted control is a bounded interpolation rather than an all-or-nothing semantic layout.

2.3 Semantic retention

Flush publishes segment metadata into the manifest and updates the degree sidecar. Reads compare the signature with this evidence before reading bodies. During compaction, a semantic merge groups outputs by useful exact partition; a naive merge may collapse keys to mixed sentinels. Unsafe reconstruction demotes the output rather than trusting it. The design does not require every stage to remain exact; it requires every skip to remain justified.

3 EVALUATION

3.1 Setup and variants

Experiments ran on one Intel Xeon 6982P-C host (64 physical cores, 128 hardware threads, 496 GiB RAM) with blocking I/O and 64 MiB memgraph/segment targets. W6 used engine freeze `324a1e2`; C2 records its stage commit `c2aa16c`. SF100 samples 5,000 deterministic stride-selected sources for edge types 1, 2, 3, 7, 8, 9, 10, 11, and 12 (45,000 operations per repeat); SF10 uses 1,000 each. There is no RNG seed. We report three in-process repeats, no explicit warmup, and no OS page-cache drop. Percentiles are means of per-edge-type percentiles across repeats, not global operation percentiles. All compared W6 rows passed count/hash checks against the same snapshot with zero mismatches.

The external artifact also contains digest-passing SF10 runs for LiveGraph, Aster RocksGraph, TuGraph, NebulaGraph, and Neo4j. Their loaders, query counts, and scopes differ from W6, so we use them only to demonstrate interface coverage and reproducibility, not to rank end-to-end systems.

3.2 Budget and resource tradeoff

Figure 2(a) shows that semantic metadata changes admission: naive and kv-lsm each accumulate 49.26M candidate-L0 admissions, versus 5.93M for schema and 6.02M for budget-64. Read-byte measurements are shown for completeness but are not a primary claim: the reader may reload whole offset arrays after metadata-cache misses, and SF100 standard deviations are large. Panel (b) likewise does not establish a stable budget-64 latency win over schema; their one-standard-deviation gate is FALLBACK.

The main budget result is the resource bound. Table 1 shows that at SF10 budget-64 stays at 13.3 GiB and 2.26 GiB RSS, while

Table 1: Implemented variants and measured import resources. Store size and L0 files are SF10; RSS is shown as SF10/SF100. Budget-64 is not expected to dominate the strong static schema baseline on every workload.

Variant	Implemented layout/admission evidence	Store GiB	L0 files	Peak RSS GiB
naive	One unsplit flush output; no intentional semantic partitioning	13.4	170	2.22 / 2.43
kv-lsm	Range-bounded KV-style order with Unknown/Mixed semantic metadata	13.4	170	2.33 / 2.49
schema	Source-label/edge-type partitions; degree remains Mixed	13.3	376	2.12 / 2.42
edge-only	Edge-type partitions; source/destination labels and degree are coarse	13.3	380	1.79 / 2.53
budg-b64	Schema base plus score-selected, degree-exact partitions under $B = 64$	13.3	444	2.26 / 2.21
semantic	All source-label/edge-type groups considered for degree-class splitting	14.6	737	8.38 / 118.03
oracle	Exact in-memory L0 index used as a pruning upper-bound reference	13.4	170	2.27 / 2.67

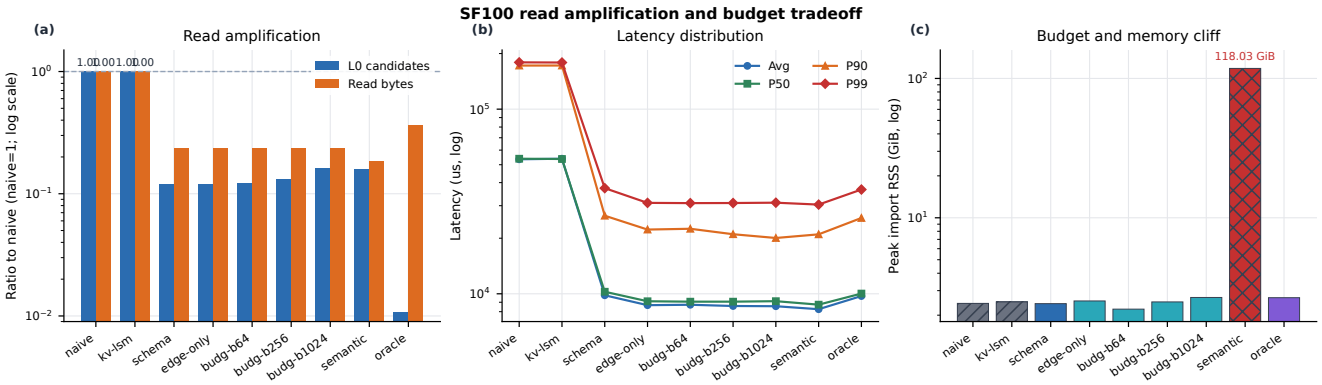

Figure 2: W6 SF100. Panel (a) divides each metric by the naive value and uses a logarithmic axis; candidate L0 is primary, while read bytes are secondary because offset-array/cache over-read produces high variance. Panel (b) is a workload-specific latency distribution; the budget-64 versus schema latency gate is FALLBACK. Panel (c) exposes the full-semantic memory cliff.

full semantic grows to 14.6 GiB and 8.38 GiB RSS. At SF100, full semantic reaches 118.03 GiB versus 2.21 GiB for budget-64. Schema is a strong static baseline; budgeted control adds a resource ceiling and a way to redirect precision. In a real-SF30-derived W7 workload shift, feedback-only control selected a hot partition in both phases and moved the selected range after the hot range changed. This is derived workload evidence, not a production trace, but it tests the mechanism that static schema lacks.

3.3 Compaction retention

For C2, the exact-surface ratio is the number of edges in topology-exact segments divided by all edges covered by the merge. Retention divides that ratio after by before. The controlled synthetic inputs deliberately construct four and six exact source-label/edge-type partitions; this isolates merge policy as the causal variable. Naive merge collapses each set to mixed output and retention 0, producing 4x/6x full-read amplification. Semantic merge keeps retention 1 and read cost 1x, with write amplification 1.87/1.85 versus 1.22/1.14 for naive.

The real SF30 run processes all 1.09 billion directed edges. Draining L0 produces the 40 observed source-label/edge-type groups and 528 exact L1 segments; 40 is therefore data-derived, not an arbitrary query count. Naive L1-to-L2 merge yields retention 0 and write amplification 1.07, while semantic merge yields retention 1 and 1.24. Panel (c) replays those 40 signatures over post-merge metadata. Naive merge admits 282.16 GB against 43.25 GB of exact partition bytes, or 6.52x. Semantic merge admits 43.25 GB, or 1.00x. This replay performs no body decode and supports no SF30 latency claim.

3.4 Lifecycle coverage and correctness

The evidence is decomposed by stage. W6 measures flush layout, admission, and resources. C2 isolates compaction retention and rewrite cost. W9 runs 30 minutes of mixed reads/writes over SF30 starting stores and records six checkpoints, but rewrite bytes are zero for all variants. It is evidence for continued flush and L0 accumulation, not concurrent compaction. W13 is a bounded ten-test suite covering stored schema epochs, fixed-width encoding changes, alias/drop handling, snapshot and tombstone visibility, reopen, and exact-versus-mixed fallback. All ten tests pass. These stages support a lifecycle-oriented design without pretending that one run exercised the full lifecycle end to end.

4 RELATED WORK

Dynamic graph stores explore transactional adjacency scans and multiversion structures [4, 5]. GraphOne and a recent DGS study examine mutable graph representations and their tradeoffs [6, 7]. LSMGraph combines dynamic updates with multi-level CSR [3]; SemL0 focuses on semantic evidence used by both read admission and physical rewrite.

BACH already uses workload and degree distribution to tune LSM-based graph format and merge choices [8]. ArceKV adapts generic LSM compaction to changing key-value workloads [9]. SemL0 is therefore not claiming the first workload-aware merge

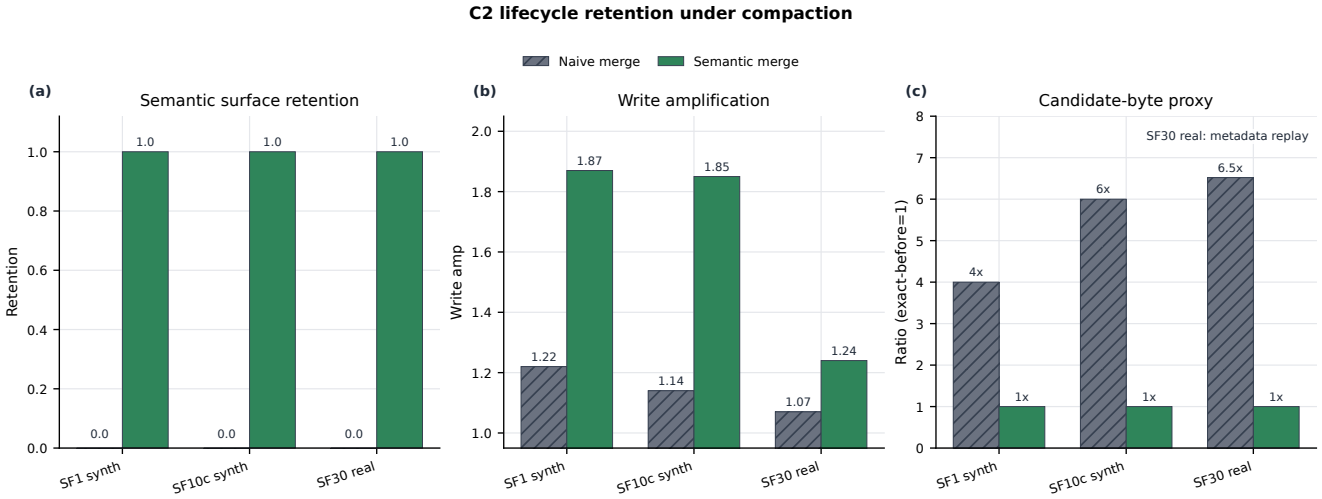

Figure 3: C2 compaction evidence. (a) retention is exact-surface ratio after divided by before. (b) write amplification is output bytes divided by logical update bytes. (c) is aggregate candidate bytes after merge divided by exact partition bytes before; SF30 is metadata replay, not body-read execution.

scheduler. Its distinction is the object being preserved: property-graph semantic evidence, a safe proof rule for skipping, and measured semantic-surface retention across compaction. Functional storage decomposition motivates separating access needs from a fixed format [10]. Graph-oriented columnar layouts make a related case for graph-specific physical organization [11].

5 LIMITATIONS AND CONCLUSION

SemL0 is a prototype, not a complete property-query engine or production graph database. Property values beyond presence remain prototype-only. SF100 read bytes carry a reader-over-read caveat, SF100 latency does not pass the budget-64/schema separation gate, W7 is real-data-derived rather than a trace, and SF30 C2 read consequence is a metadata proxy. Schema/snapshot evidence is a bounded correctness suite, not general online migration.

Within those boundaries, the result is consistent: graph query semantics can be persistent storage evidence; every skip must follow an exactness contract; the evidence needs a file/resource budget; and compaction must explicitly retain or demote it. Treating compaction as semantic rewrite is the central systems lesson.

REFERENCES

[1] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil. The log-structured merge-tree. *Acta Informatica*, 33(4):351–385, 1996.
[2] O. Erling et al. The LDBC social network benchmark: Interactive workload. In *SIGMOD*, 2015.
[3] S. Yu et al. LSMGraph: A high-performance dynamic graph storage system with multi-level CSR. *Proc. ACM Manag. Data*, 2(6), Article 243, 2024.
[4] X. Zhu et al. LiveGraph: A transactional graph storage system with purely sequential adjacency list scans. *PVLDB*, 13(7):1020–1034, 2020.
[5] D. De Leo and P. Boncz. Teseo and the analysis of structural dynamic graphs. *PVLDB*, 14(6):1053–1066, 2021.
[6] P. Kumar and H. H. Huang. GraphOne: A data store for real-time analytics on evolving graphs. In *FAST*, 2019.
[7] J. Su et al. Revisiting the design of in-memory dynamic graph storage. *Proc. ACM Manag. Data*, 3(1), Article 70, 2025. DOI: 10.1145/3709720.
[8] J. Huang, Y. Cao, S. Ren, B. Wu, and D. Miao. BACH: Bridging adjacency list and CSR format using LSM-trees for HGTAP workloads. *PVLDB*, 18(5):1509–1521, 2025.
[9] J. Liu, H. Xie, and S. Luo. ArceKV: Towards workload-driven LSM-compactions for key-value store under dynamic workloads. *PVLDB*, 19(5):958–972, 2026.
[10] M. Prammer et al. Towards functional decomposition of storage formats. In *CIDR*, 2025.
[11] P. Gupta, A. Mhedhbi, and S. Salihoglu. Columnar storage and list-based processing for graph database management systems. *PVLDB*, 14(11):2491–2504, 2021.